

Dense Times Random / Mixed Precision Matrix Multiplication Tiling Schemes

April 2021

1 Introduction

Claim: a nearly identical tiling scheme and analysis to the “recompute-when-multiplying by a random matrix” also gives a better tiling scheme for mixed precision matrix multiplication $C = A \cdot B$, where A is stored at a lower precision than either B or C . The intuition is the same: we can get away with loading the cheaper matrix multiple times over while reducing the number of times we load the more expensive matrix. This is interesting since it relies on the input matrices having asymmetric costs to access, as opposed to previous analysis which keeps only the output matrix C in higher precision (in which case, we still get square tiles of C since the cost to access the input matrices are identical).

The analysis below closely mirrors the paper <https://arxiv.org/pdf/1702.02017.pdf> (that treats standard DGEMM, nothing fancy) and the analysis already conducted for on-the-fly matrix multiplication. I think that this would be another very interesting use case for this tiling scheme, with applications to training neural networks in low precision, etc. The overall strategy is blocked matrix multiplication where we keep a large block of only the output matrix C in fast-memory while streaming in rows / columns of A, B . Section 4.3 of their paper uses the large-block-of- C -in-memory approach and achieve a cost of

$$\frac{2mnk}{\sqrt{M}} + mn$$

reads and mn writes for performing standard matrix multiplication of A, B, C (the dimensions m, n, k are detailed below).

2 Faster Tiling Scheme for Mixed Precision matrix Multiplication

Suppose we want to compute

$$C = A \cdot B$$

where $C \in R^{m \times n}$, $A \in R^{m \times k}$, $B \in R^{k \times n}$. Further suppose that C and B are stored in “full” precision, but A is stored at a lower precision. Let ρ be the ratio of the number of bits required to store an entry of A compared to an entry of B or C (for A at 8-bit precision and C at 64-bit precision, we have $\rho = 1/8$). Assume that each entry of either B or C fits into 1 “word”, so to bring w entries of C into fast-memory, we have to move w words; but to access w entries of A , we only have to bring ρw words into fast memory.

Take z as a tuning parameter and partition the three matrices as follows:

- C into a 2D grid of blocks of dimension $\approx z \times M/z$ (may have to subtract 1 from each dimension, but this is the rough shape).
- A by contiguous chunks of rows; each chunk has dimensions $z \times k$.
- B by contiguous chunks of columns; each chunk has dimensions $k \times M/z$.

At any given time, we will keep a block of C in fast-memory while streaming in parts of A, B . We have $\frac{m}{z} \times \frac{n}{M/z} = \frac{mn}{M}$ blocks of C that we need to process, and for each block of C , we read

$$\rho z k + k(M/z) + M$$

words (each term coming from A, B , or C respectively). We perform mn writes in total across the kernel, one write per element of C .

Now the goal is to choose z to minimize $\rho z + (M/z)$ (strip away the constant factor of k that multiplies this quantity). We get the condition $\rho = \frac{M}{z^2}$ which yields $z = \sqrt{M/\rho}$. Notice for $\rho = 1$, this yields a square tile shape, as expected. Substituting, the final memory movement cost is

$$\frac{mnk}{M} \left(2\sqrt{M\rho} \right) + mn$$

which we simplify to get

$$2 \frac{mnk\sqrt{\rho}}{\sqrt{M}} + mn$$

What happens if $\rho < 1$, but we use square tiles anyway (i.e. we don’t modify the strategy for DGEMM)? Then we get a number of reads

$$\frac{(1 + \rho)mnk}{\sqrt{M}} + mn$$

from the simple fact that loading elements from A costs less than loading elements from B or C . The number of writes stays the same. Observe that $(1 - \sqrt{\rho})^2 = 1 + \rho - 2\sqrt{\rho} > 0$ (since $\rho < 1$), so choosing $z = \sqrt{M/\rho}$ always wins (or does just as well as) picking the square tiling scheme, and it moves

$\frac{(1-\sqrt{\rho})^2 mnk}{\sqrt{M}}$ fewer words than the standard DGEMM tiling scheme.

Let's check the speedup, assuming that B, C are stored in double (64-bit) precision (and accounting for the fact that moving an entry of A is cheaper than moving a double-word):

- For $\rho = 1$, no advantage (we degenerate to square tiles for C).
- For $\rho = 1/2$ (float precision for A), we move 90% of the words than if we used a square tile for C .
- For $\rho = 1/4$ (Bfloat16, for example), we move 75% of the words...
- For $\rho = 1/8$ (I have seen one paper that attempts to train neural networks with 8-bit precision), we move 58% of the words....
- For $\rho = 1/64$ (so, single-bit matrices), the number of reads is 23% that of the square tiling strategy for C .

3 Intuition

I understand that there have been attempts to compute C in a precision higher than either matrix A or B . Since the cost to access A and B is the same, however, symmetrically the best tile shape is a square. The change above happens if we want a write-efficient algorithm with an asymmetric cost to access either A vs. B .

The intuition is the same as recomputation when multiplying by a random matrix: notice that as ρ gets lower, z gets higher, so we are performing more memory loads of the cheaper A matrix in comparison to the B matrix (the same way that we recompute elements of the A matrix-on-the-fly).

4 Applications

Training neural networks in low precision, etc.